



Celebal Summer Internship 2026

Week 3 Task: Subqueries, CTE's and Window Functions

Tasks

Step 1: Setup Data :-

Q1.Import the Superstore dataset into a table named **superstore_raw**.

1. Opened MySQL Workbench and connected to the database.
2. Created a database named `intern`.
3. Selected the database using:

```
USE intern;
```

4. Used the **Table Data Import Wizard** to import the Superstore CSV file.
5. Imported the dataset into a new table named **superstore_raw**.

Q2.Create these 3 tables from it:

- a. customers
- b. orders
- c. products

```
-- Create these 3 tables from it:
```



```
1. customers:-
```

```
CREATE TABLE customers (  
    Customer_ID VARCHAR(20),  
    Customer_Name VARCHAR(100),  
    segment VARCHAR(50),  
    country VARCHAR(50),  
    city VARCHAR(50),  
    state VARCHAR(50),  
    Postal_Code VARCHAR(20),  
    region VARCHAR(50)  
);
```

Created Table: Customers

2.products:-

```
CREATE TABLE products (  
    Product_ID VARCHAR(30),  
    category VARCHAR(50),  
    Sub_Category VARCHAR(50),  
    Product_Name VARCHAR(255)  
);
```

Created Table: Products

3.orders:-

```
CREATE TABLE orders (  
    Row_ID INT,  
    Order_ID VARCHAR(30),  
    Order_Date DATE,  
    Ship_Date DATE,  
    Ship_Mode VARCHAR(50),  
    Customer_ID VARCHAR(20),  
    Product_ID VARCHAR(30),  
    sales DECIMAL(10,2),  
    quantity INT,  
    discount DECIMAL(5,2),  
    profit DECIMAL(10,2)  
);
```

Created Table: Orders

Q3. Insert data into these tables using **SELECT DISTINCT**.

```
55 1. For customers:-
56
57 INSERT INTO customers
58 SELECT DISTINCT
59     Customer_ID,
60     Customer_Name,
61     segment,
62     country,
63     city,
64     state,
65     Postal_Code,
66     region
67 FROM superstore_raw;
68 -- To show data --
69 • select * from customers;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	Customer_ID	Customer_Name	segment	country	city	state	Postal_Code	region
▶	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South
	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West
	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South
	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West
	AA-10480	Andrew Allen	Consumer	United States	Concord	North Carolina	28027	South
	IM-15070	Irene Maddox	Consumer	United States	Seattle	Washington	98103	West

Data inserted into table Customers using Distinct

```
70
71 2. for products:-
72
73 INSERT INTO products
74 SELECT DISTINCT
75     Product_ID,
76     Category,
77     Sub_Category,
78     Product_Name
79 FROM superstore_raw;
80 • select * from products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	Product_ID	category	Sub_Category	Product_Name
	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase
	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...
	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...
	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table
	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System
	FUR-FU-10001487	Furniture	Furnishings	Eldon Expressions Wood and Plastic Desk Acces...

Data inserted into table_products using Distinct

Step 2: Perform Required Queries :-

Q1. Find all orders where sales are greater than the average sales. (Subquery)

```
98
99  -- Step 2: Perform Required Queries --
100
101  -- 1.Find all orders where sales are greater than the average sales. (Subquery) --
102 • SELECT *
103    FROM orders
104   WHERE sales > (
105       SELECT AVG(sales)
106       FROM orders
107   );
108
```

Result Grid

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Product_ID	sales	quantity	discount	profit
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	FUR-BO-10001798	261.96	2	0.00	41.91
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.58
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.03
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	TEC-PH-10002275	907.15	6	0.20	90.72
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	FUR-TA-10001539	1706.18	9	0.20	85.31

orders 49 x

Q2. Find the highest sales order for each customer. (Subquery)

```
108
109  -- 2.Find the highest sales order for each customer. (Subquery) --
110 • SELECT * FROM orders o
111   WHERE sales = (
112       SELECT MAX(sales)
113       FROM orders
114       WHERE Customer_ID = o.Customer_ID
115   ) limit 5;
116
117
```

Result Grid

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Product_ID	sales	quantity	discount	profit
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.58
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.03
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	FUR-TA-10001539	1706.18	9	0.20	85.31
25	CA-2015-106320	2015-09-25	2015-09-30	Standard Class	EB-13870	FUR-TA-10000577	1044.63	3	0.00	240.26
28	US-2015-150630	2015-09-17	2015-09-21	Standard Class	TB-21520	FUR-BO-10004834	3083.43	7	0.50	-1665.05

orders 50 x

Q3. Calculate total sales for each customer. (CTE)

```
117
118 -- 3.Calculate total sales for each customer. (CTE) --
119 • WITH customer_sales AS
120 (
121     SELECT
122     customer_id,
123     SUM(sales) AS total_sales
124     FROM orders
125     GROUP BY customer_id
126 )
127 SELECT *
128 FROM customer_sales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	total_sales
▶	CG-12520	4595.12
	DV-13045	4477.92
	SO-20335	10410.32
	BH-11710	25021.36
	AA-10480	7131.48
	IM-15070	19721.96
	HP-14815	7961.24
	PK-19075	32634.60

Result 51 x

Q4. Find customers whose total sales are above average. (CTE + Subquery)

```
131 • WITH customer_sales AS(
132     SELECT
133     customer_id,
134     SUM(sales) AS total_sales
135     FROM orders
136     GROUP BY customer_id
137 )
138 SELECT *
139 FROM customer_sales
140 WHERE total_sales > (
141     SELECT AVG(total_sales)
142     FROM customer_sales
143 );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	total_sales
▶	BH-11710	25021.36
	IM-15070	19721.96
	PK-19075	32634.60
	TB-21520	18922.48
	MA-17560	17122.48
	SN-20710	12511.88
	LC-16930	17924.64
	RA-19885	15329.28

Result 52 x

Q5. Rank all customers based on total sales. (*Window Function*)

```
144
145 -- 5.Rank all customers based on total sales. (Window Function)--
146 • WITH customer_sales AS(
147     SELECT
148         customer_id,
149         SUM(sales) AS total_sales
150     FROM orders
151     GROUP BY customer_id
152 )SELECT
153     customer_id,
154     total_sales,
155     RANK() OVER(ORDER BY total_sales DESC) AS sales_rank
156 FROM customer_sales;
157
```

customer_id	total_sales	sales_rank
SM-20320	100172.28	1
TC-20980	76071.40	2
RB-19360	60469.40	3
TA-21385	58382.48	4
AB-10105	57422.44	5

Q6. Assign row numbers to each order within a customer. (*Window Function + PARTITION BY*)

```
157
158 -- 6.Assign row numbers to each order within a customer. (Window Function + PARTITION BY) --
159 • SELECT
160     customer_id,
161     order_id,
162     sales,
163     ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY sales DESC) AS row_num
164 FROM orders;
165
```

customer_id	order_id	sales	row_num
AA-10315	CA-2016-103982	3930.07	1
AA-10315	CA-2016-103982	3930.07	2
AA-10315	CA-2016-103982	3930.07	3
AA-10315	CA-2016-103982	3930.07	4
AA-10315	CA-2014-128055	673.57	5
AA-10315	CA-2014-128055	673.57	6
AA-10315	CA-2014-128055	673.57	7
AA-10315	CA-2014-128055	673.57	8
AA-10315	CA-2016-103982	431.98	9
AA-10315	CA-2016-103982	431.98	10
AA-10315	CA-2016-103982	431.98	11
AA-10315	CA-2016-103982	431.98	12

Q7. Display top 3 customers based on total sales. (Window Function)

```
165
166 -- 7.Display top 3 customers based on total sales. (Window Function) --
167 WITH customer_sales AS(
168     SELECT
169         customer_id,
170         SUM(sales) AS total_sales
171     FROM orders
172     GROUP BY customer_id
173 )SELECT *FROM(
174     SELECT
175         customer_id,
176         total_sales,
177         RANK() OVER(ORDER BY total_sales DESC) AS sales_rank FROM customer_sales) ranked_customers
178 WHERE sales_rank <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	total_sales	sales_rank
▶	SM-20320	100172.28	1
	TC-20980	76071.40	2
	RB-19360	60469.40	3

Step 3: Final Combined Query :-

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

(Use **JOIN + CTE + Window Function** together)

```
187
188 • WITH customer_sales AS(
189     SELECT
190         c.customer_name,
191         SUM(o.sales) AS total_sales
192     FROM customers c
193     JOIN orders o
194     ON c.customer_id = o.customer_id
195     GROUP BY c.customer_name
196 )SELECT
197     customer_name,
198     total_sales,RANK() OVER(ORDER BY total_sales DESC) AS sales_rank FROM customer_sales;
199
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_name	total_sales	sales_rank
Sanjit Engle	1074430.72	1
Clay Ludtke	1044533.76	2
Sanjit Chand	1018248.48	3
Ken Lonsdale	1013178.24	4
Sean Miller	1001722.80	5

result 56 x

Mini Project: Customer Sales Insights :-

Q1. Who are the top 5 customers?

```
199
200  -- Mini Project: Customer Sales Insights --
201
202  -- 1. Who are the top 5 customers? --
203 • SELECT
204      c.customer_name,
205      SUM(o.sales) AS total_sales
206  FROM customers c
207  JOIN orders o
208  ON c.customer_id = o.customer_id
209  GROUP BY c.customer_name
210  ORDER BY total_sales DESC
211  LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	customer_name	total_sales
▶	Sanjit Engle	1074430.72
	Clay Ludtke	1044533.76
	Sanjit Chand	1018248.48
	Ken Lonsdale	1013178.24
	Sean Miller	1001722.80

Result 57

Q2. Who are the bottom 5 customers?

```
212
213  -- 2. Who are the bottom 5 customers? --
214 • SELECT
215      c.customer_name,
216      SUM(o.sales) AS total_sales
217  FROM customers c
218  JOIN orders o
219  ON c.customer_id = o.customer_id
220  GROUP BY c.customer_name
221  ORDER BY total_sales ASC
222  LIMIT 5;
223
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch r

	customer_name	total_sales
▶	Lela Donovan	42.40
	Thais Sissman	77.44
	Mitch Gastineau	98.56
	Carl Jackson	132.16
	Roy Skaria	357.28

Result 58

Q3. Which customers made only one order?

```
223
224 -- 3.Which customers made only one order? --
225 • SELECT
226     customer_id,
227     COUNT(order_id) AS total_orders
228 FROM orders
229 GROUP BY customer_id
230 HAVING COUNT(order_id) = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	total_orders
-------------	--------------

There is no customer that made only one order. It simply means that in this dataset, every customer has placed more than one order.

Q4. Which customers have above-average sales?

```
231
232 -- 4.Which customers have above-average sales? --
233 • WITH customer_sales AS(
234     SELECT
235         customer_id,
236         SUM(sales) AS total_sales
237     FROM orders
238     GROUP BY customer_id
239 )
240 SELECT *
241 FROM customer_sales
242 WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales);
243
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	total_sales
BH-11710	25021.36
IM-15070	19721.96
PK-19075	32634.60
TB-21520	18922.48
MA-17560	17122.48

Result 61 x

Q5. What is the highest order value per customer?

```
243
244 -- 5.What is the highest order value per customer? --
245 • SELECT
246     customer_id,
247     MAX(sales) AS highest_order_value
248 FROM orders
249 GROUP BY customer_id;
```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	customer_id	highest_order_value
▶	CG-12520	731.94
	DV-13045	721.88
	SO-20335	957.58
	BH-11710	1706.18
	AA-10480	479.97

Result 62 ×